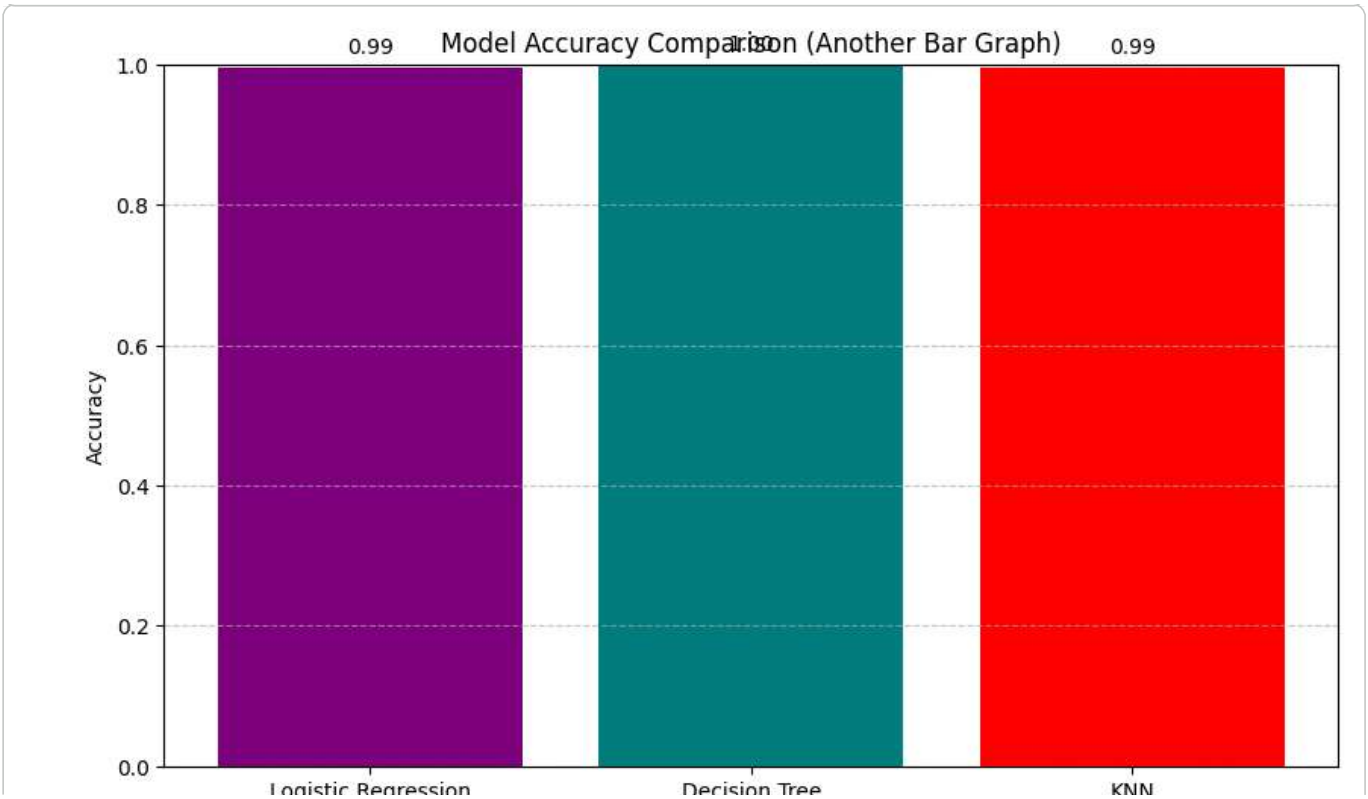
```
# Model Accuracy Comparison for Guess Game
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# --- Generate synthetic dataset ---
# Feature: guesses (1–100), Target: outcome (0=too low, 1=too high, 2=correct)
np.random.seed(42)
X = np.random.randint(1, 101, (200, 1))
y = []
secret_number = 50
for guess in X:
    if guess < secret_number:
        y.append(0)
    elif guess > secret_number:
        y.append(1)
    else:
        y.append(2)
y = np.array(y)

# --- Train models ---
lr = LogisticRegression(max_iter=1000).fit(X, y)
dt = DecisionTreeClassifier().fit(X, y)
knn = KNeighborsClassifier(n_neighbors=5).fit(X, y)

# --- Predictions ---
y_pred_lr = lr.predict(X)
y_pred_dt = dt.predict(X)
y_pred_knn = knn.predict(X)

# --- Accuracies ---
lr_acc = accuracy_score(y, y_pred_lr)
dt_acc = accuracy_score(y, y_pred_dt)
knn_acc = accuracy_score(y, y_pred_knn)

# --- Visualization ---
models = ["Logistic Regression", "Decision Tree", "KNN"]
accuracies = [lr_acc, dt_acc, knn_acc]

plt.figure(figsize=(8,5))
plt.bar(models, accuracies, color=["blue","green","orange"])
plt.title("Model Accuracy Comparison for Guess Game")
plt.ylabel("Accuracy")
plt.ylim(0,1)
for i, acc in enumerate(accuracies):
    plt.text(i, acc+0.02, f"{acc:.2f}", ha='center')
plt.show()

# --- Print results ---
print("Logistic Regression Accuracy:", lr_acc)
print("Decision Tree Accuracy:", dt_acc)
print("KNN Accuracy:", knn_acc)
```

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# --- Generate synthetic dataset --- (Copied from DE21BwJ0M42n for context)
# Feature: guesses (1–100), Target: outcome (0=too low, 1=too high, 2=correct)
```

```
# Feature: guesses (1-100), Target: outcome (0-too low, 1-too high, 2-correct)
np.random.seed(42)
X = np.random.randint(1, 101, (200, 1))
y = []
secret_number = 50
for guess in X:
    if guess < secret_number:
        y.append(0)
    elif guess > secret_number:
        y.append(1)
    else:
        y.append(2)
y = np.array(y)

# --- Train models --- (Copied from DE21BwJ0M42n for context)
lr = LogisticRegression(max_iter=1000).fit(X, y)
dt = DecisionTreeClassifier().fit(X, y)
knn = KNeighborsClassifier(n_neighbors=5).fit(X, y)

# --- Predictions --- (Copied from DE21BwJ0M42n for context)
y_pred_lr = lr.predict(X)
y_pred_dt = dt.predict(X)
y_pred_knn = knn.predict(X)

# --- Accuracies --- (Copied from DE21BwJ0M42n for context)
lr_acc = accuracy_score(y, y_pred_lr)
dt_acc = accuracy_score(y, y_pred_dt)
knn_acc = accuracy_score(y, y_pred_knn)

# --- Visualization Data --- (Copied from DE21BwJ0M42n for context)
models = ["Logistic Regression", "Decision Tree", "KNN"]
accuracies = [lr_acc, dt_acc, knn_acc]

# --- Another Bar Graph for Visualization ---
plt.figure(figsize=(10, 6))
plt.bar(models, accuracies, color=['purple', 'teal', 'red'])
plt.title('Model Accuracy Comparison (Another Bar Graph)')
plt.xlabel('Model')
plt.ylabel('Accuracy')
plt.ylim(0, 1)
for i, acc in enumerate(accuracies):
    plt.text(i, acc + 0.02, f'{acc:.2f}', ha='center')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```


Model Accuracy Comparison (Another Bar Graph)
0.99
1.00
0.99
1.0
0.8
0.6
0.4
0.2
0.0
Accuracy
Logistic Regression
Decision Tree
KNN